

A stochastic algorithm for bilevel optimization with a mixed-integer upper-level problem and applications to neural architecture search

I. Garee T. Giovannelli

Department of Mechanical, Materials, and Industrial Engineering
University of Cincinnati, Cincinnati, OH

Modeling and Optimization: Theory and Applications (MOPTA)
August 19, 2026



Outline

- 1 Introduction and motivation
- 2 Background: Bilevel stochastic gradient
- 3 Background: Derivative-free optimization with primitive directions
- 4 The BSGDFL algorithm
- 5 Application to neural architecture search

Outline

- 1 Introduction and motivation
- 2 Background: Bilevel stochastic gradient
- 3 Background: Derivative-free optimization with primitive directions
- 4 The BSGDFL algorithm
- 5 Application to neural architecture search

Bilevel Optimization with a Mixed-Integer Upper-Level Problem

$$\begin{aligned} \min_{x \in \mathbb{R}^n, y \in \mathbb{R}^m} \quad & f_u(x, y) \\ \text{s.t.} \quad & x \in X \cap \mathcal{Z} \\ & y \in \arg \min_{y \in \mathbb{R}^m} f_\ell(x, y) \end{aligned}$$

where f_u and f_ℓ are the **upper-level** (UL) and **lower-level** (LL) objective functions, resp.

Bilevel Optimization with a Mixed-Integer Upper-Level Problem

$$\begin{aligned} \min_{x \in \mathbb{R}^n, y \in \mathbb{R}^m} \quad & f_u(x, y) \\ \text{s.t.} \quad & x \in X \cap \mathcal{Z} \\ & y \in \arg \min_{y \in \mathbb{R}^m} f_\ell(x, y) \end{aligned}$$

where f_u and f_ℓ are the **upper-level** (UL) and **lower-level** (LL) objective functions, resp.

$I_c, I_z \subseteq \{1, \dots, n\}$ are index sets of continuous and integer **UL** variables, resp.

$X = \{x \in \mathbb{R}^n \mid \ell \leq x \leq u\}$ and $\mathcal{Z} = \{x \in \mathbb{R}^n \mid x_i \in \mathbb{Z} \text{ for } i \in I_z\}$

Bilevel Optimization with a Mixed-Integer Upper-Level Problem

$$\begin{aligned}
 \min_{x \in \mathbb{R}^n, y \in \mathbb{R}^m} \quad & f_u(x, y) \\
 \text{s.t.} \quad & x \in X \cap \mathcal{Z} \\
 & y \in \arg \min_{y \in \mathbb{R}^m} f_\ell(x, y)
 \end{aligned}$$

where f_u and f_ℓ are the **upper-level** (UL) and **lower-level** (LL) objective functions, resp.

$I_c, I_z \subseteq \{1, \dots, n\}$ are index sets of continuous and integer **UL** variables, resp.

$X = \{x \in \mathbb{R}^n \mid \ell \leq x \leq u\}$ and $\mathcal{Z} = \{x \in \mathbb{R}^n \mid x_i \in \mathbb{Z} \text{ for } i \in I_z\}$

$x = (x_c, x_z)$, with $x_c = (x_i \mid i \in I_c)$ and $x_z = (x_i \mid i \in I_z)$

Bilevel Optimization with a Mixed-Integer Upper-Level Problem

$$\begin{aligned} \min_{x \in \mathbb{R}^n, y \in \mathbb{R}^m} \quad & f_u(x, y) \\ \text{s.t.} \quad & x \in X \cap \mathcal{Z} \\ & y \in \arg \min_{y \in \mathbb{R}^m} f_\ell(x, y) \end{aligned}$$

Additional challenges:

f_u and f_ℓ involve expectations or large finite sums, requiring **stochastic approximations**.

Bilevel Optimization with a Mixed-Integer Upper-Level Problem

$$\begin{aligned}
 \min_{x \in \mathbb{R}^n, y \in \mathbb{R}^m} \quad & f_u(x, y) \\
 \text{s.t.} \quad & x \in X \cap \mathcal{Z} \\
 & y \in \arg \min_{y \in \mathbb{R}^m} f_\ell(x, y)
 \end{aligned}$$

Additional challenges:

f_u and f_ℓ involve expectations or large finite sums, requiring **stochastic approximations**.

f_u is **black-box** (unknown analytical expression) in x_z

Bilevel Optimization with a Mixed-Integer Upper-Level Problem

$$\begin{aligned}
 \min_{x \in \mathbb{R}^n, y \in \mathbb{R}^m} \quad & f_u(x, y) \\
 \text{s.t.} \quad & x \in X \cap \mathcal{Z} \\
 & y \in \arg \min_{y \in \mathbb{R}^m} f_\ell(x, y)
 \end{aligned}$$

Additional challenges:

f_u and f_ℓ involve expectations or large finite sums, requiring **stochastic approximations**.

f_u is **black-box** (unknown analytical expression) in x_z

The integrality of x_z cannot be relaxed.

Single-Level Reformulation

$$\begin{aligned} \min_{x \in \mathbb{R}^n, y \in \mathbb{R}^m} \quad & f_u(x, y) \\ \text{s.t.} \quad & x \in X \cap \mathcal{Z} \\ & y \in \arg \min_{y \in \mathbb{R}^m} f_\ell(x, y) \end{aligned}$$

We assume the **LL problem** admits a unique solution, i.e., $\exists! y(x) \ \forall x \in X \cap \mathcal{Z}$

Single-Level Reformulation

$$\begin{aligned} \min_{x \in \mathbb{R}^n, y \in \mathbb{R}^m} \quad & f_u(x, y) \\ \text{s.t.} \quad & x \in X \cap \mathcal{Z} \\ & y \in \arg \min_{y \in \mathbb{R}^m} f_\ell(x, y) \end{aligned}$$

We assume the **LL problem** admits a unique solution, i.e., $\exists! y(x) \quad \forall x \in X \cap \mathcal{Z}$

The bilevel problem can be posed solely in the **UL variables**

$$\min_{x \in \mathbb{R}^n} f(x) = f_u(x, y(x)) \quad \text{s.t.} \quad x \in X \cap \mathcal{Z}$$

This is a single-level mixed-integer black-box optimization problem.

Motivation: Neural Architecture Search (NAS)

Goal: Automate the task of **designing** Deep Neural Networks (DNNs) by optimizing architectural and training choices to improve generalization performance.

Motivation: Neural Architecture Search (NAS)

Goal: Automate the task of **designing** Deep Neural Networks (DNNs) by optimizing architectural and training choices to improve generalization performance.

UL problem: Finds the values of the DNN hyperparameters x to minimize the **validation** error.

x_c : Learning rate and regularization coefficients (continuous)

x_z : Model depth, width, activation functions, operation choices (integer/categorical)

Motivation: Neural Architecture Search (NAS)

Goal: Automate the task of **designing** Deep Neural Networks (DNNs) by optimizing architectural and training choices to improve generalization performance.

UL problem: Finds the values of the DNN hyperparameters x to minimize the **validation** error.

x_c : Learning rate and regularization coefficients (continuous)

x_z : Model depth, width, activation functions, operation choices (integer/categorical)

(The validation error is **black-box** w.r.t. the learning rate and x_z)

Motivation: Neural Architecture Search (NAS)

Goal: Automate the task of **designing** Deep Neural Networks (DNNs) by optimizing architectural and training choices to improve generalization performance.

UL problem: Finds the values of the DNN hyperparameters x to minimize the **validation** error.

x_c : Learning rate and regularization coefficients (continuous)

x_z : Model depth, width, activation functions, operation choices (integer/categorical)

(The validation error is **black-box** w.r.t. the learning rate and x_z)

LL problem: Finds the values of the DNN weights y to minimize the **training** error.

Motivation: Neural Architecture Search (NAS) - Our Setting

Goal: Automate the task of **designing** Deep Neural Networks (DNNs) by optimizing architectural and training choices to improve generalization performance.

UL problem: Finds the values of the DNN hyperparameters x to minimize the **validation** error.

x_c : ~~Learning rate~~ and regularization coefficients (continuous)

x_z : Model depth, width, ~~activation functions, operation choices~~ (integer/categorical)

(The validation error is **black-box** w.r.t. the learning rate and x_z)

In this work: We focus on regularization coefficients, model depth, and model width.

Bilevel Formulation for NAS

Let D_{train} and D_{val} be training and validation sets, respectively, containing input-response pairs (z, u) , and let $\ell(\phi(z; \mathbf{x}, \mathbf{y}), u)$ be a loss function. Assume $\mathbf{x}_c = (\dots, \lambda)$.

$$\begin{aligned} \min_{(\mathbf{x}, \mathbf{y})} \quad & \frac{1}{|D_{\text{val}}|} \sum_{(z, u) \in D_{\text{val}}} \ell(\phi(z; \mathbf{x}, \mathbf{y}), u) \\ \text{s.t.} \quad & \mathbf{y} \in \arg \min_{\mathbf{y}} \frac{1}{|D_{\text{train}}|} \sum_{(z, u) \in D_{\text{train}}} \ell(\phi(z; \mathbf{x}, \mathbf{y}), u) + \lambda \|\mathbf{y}\|_2^2 \end{aligned}$$

Bilevel Formulation for NAS

Let D_{train} and D_{val} be training and validation sets, respectively, containing input-response pairs (z, u) , and let $\ell(\phi(z; \mathbf{x}, \mathbf{y}), u)$ be a loss function. Assume $\mathbf{x}_c = (\dots, \lambda)$.

$$\begin{aligned} \min_{(\mathbf{x}, \mathbf{y})} \quad & \frac{1}{|D_{\text{val}}|} \sum_{(z, u) \in D_{\text{val}}} \ell(\phi(z; \mathbf{x}, \mathbf{y}), u) \\ \text{s.t.} \quad & \mathbf{y} \in \arg \min_{\mathbf{y}} \frac{1}{|D_{\text{train}}|} \sum_{(z, u) \in D_{\text{train}}} \ell(\phi(z; \mathbf{x}, \mathbf{y}), u) + \lambda \|\mathbf{y}\|_2^2 \end{aligned}$$

The objective functions are estimated using mini-batches sampled from D_{train} and D_{val}

Research question: How can we design a stochastic algorithm for bilevel problems with mixed-integer UL variables and a UL objective that is black-box in the integer variables?

Where this paper fits

Bilevel stochastic gradient (BSG)

Giovanelli, Kent, and Vicente, 2025

Stochastic gradient method for bilevel optimization with continuous UL variables and inexact LL solutions

Derivative-free nonsmooth - Derivative-free line search (DFNDFL)

Giovanelli, Liuzzi, Lucidi, and Rinaldi, 2022

Derivative-free method for black-box mixed-integer optimization using line searches on:

- random directions for continuous variables (DFN)
- primitive directions for integer variables (DFL)

BSGDFL (this paper)

Stochastic alternating method for bilevel optimization with mixed-integer UL variables:

- BSG for continuous UL variables
- DFL for integer UL variables

Case study

Neural architecture search (NAS)

Related Work (NAS and Bilevel Optimization)

Approach	Limitation
Continuous relaxation of discrete variables followed by rounding	DARTS (Liu et al., 2019) introduced a softmax relaxation over candidate operations, enabling gradient-based optimization through a mixture of operations (see next slide). Convergence guarantees apply only to the relaxed problem , not the rounded architecture.
Reinforcement learning / evolutionary algorithms / Bayesian optimization	Handle discrete choices directly, but generally lack convergence guarantees .

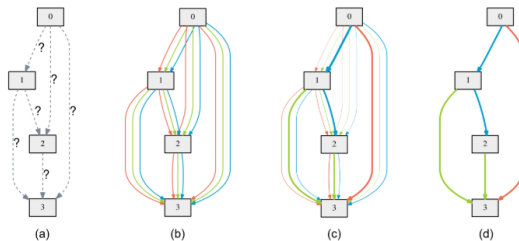
Venturini et al. (2025) proposes a continuous bilevel reformulation for a mixed-**binary** UL problem, which **does not apply** to the black-box mixed-integer NAS setting considered here.

Gap: Bilevel methods that handle mixed-integer UL variables and preserve convergence guarantees.

Continuous Relaxation of the Search Space in NAS

In Liu et al. (2019), architecture choices are represented as continuous variables rather than discrete ones.

A computation cell is a basic building block of an architecture.



Each pair of nodes is associated with a set of candidate operations (e.g., convolution, pooling, skip, etc.), parameterized through real-valued weights.

Continuous Relaxation of the Search Space in NAS

The output of a computation cell is obtained by applying the **softmax function** over the set of candidate operations $\mathcal{O}$.

Specifically, the output of the cell between nodes i and j applied to input x is:

$$o^{(i,j)}(x) = \sum_{o \in \mathcal{O}} \frac{\exp(\alpha_o^{(i,j)})}{\sum_{o' \in \mathcal{O}} \exp(\alpha_{o'}^{(i,j)})} o(x)$$

The weights $\alpha_o^{(i,j)}$ are treated as **UL** variables.

Related Work (Derivative-Free Optimization)

Method

Giovannelli et al. (2022)

Setting

Black-box mixed-integer nonsmooth optimization.

Alternates between:

- random directions for continuous variables
- primitive directions for integer variables

No relaxation of integer variables.

Lapucci et al. (2026)

The objective function is black-box only w.r.t. the discrete variables.

Continuous variables are handled through gradient-based line searches.

Gap: Extend derivative-free methods from single-level to bilevel mixed-integer problems.

Outline

- 1 Introduction and motivation
- 2 Background: Bilevel stochastic gradient
- 3 Background: Derivative-free optimization with primitive directions
- 4 The BSGDFL algorithm
- 5 Application to neural architecture search

Bilevel Optimization Problem

$$\begin{aligned} \min_{x \in \mathbb{R}^n, y \in \mathbb{R}^m} \quad & f_u(x, y) \\ \text{s.t.} \quad & x \in X \\ & y \in \arg \min_{y \in \mathbb{R}^m} f_\ell(x, y) \end{aligned}$$

where X is assumed closed and convex.

We assume the **LL problem** is **well-defined**, i.e., $\exists! y(x) \forall x \in X$

Bilevel Optimization Problem

$$\begin{aligned} \min_{x \in \mathbb{R}^n, y \in \mathbb{R}^m} \quad & f_u(x, y) \\ \text{s.t.} \quad & x \in X \\ & y \in \arg \min_{y \in \mathbb{R}^m} f_\ell(x, y) \end{aligned}$$

where X is assumed closed and convex.

We assume the **LL problem** is **well-defined**, i.e., $\exists! y(x) \forall x \in X$

The bilevel problem can be reduced to

$$\min_{x \in X} f(x) = f_u(x, y(x))$$

The bilevel stochastic gradient (BSG) method applies SG to this single-level reformulation.

Computing the Adjoint Gradient (∇f)

$\nabla f(x)$ can be computed by applying the **chain rule** to $f_u(x, y(x))$

$$\nabla f(x) = \nabla_x f_u(x, y(x)) + \nabla y(x) \nabla_y f_u(x, y(x))$$

where $\nabla y(x)$ is the Jacobian of $y(x)$, typically **unknown**.

Computing the Adjoint Gradient (∇f)

$\nabla f(x)$ can be computed by applying the **chain rule** to $f_u(x, y(x))$

$$\nabla f(x) = \nabla_x f_u(x, y(x)) + \nabla y(x) \nabla_y f_u(x, y(x))$$

where $\nabla y(x)$ is the Jacobian of $y(x)$, typically **unknown**.

The **LL 1st-order optimality condition** is $\nabla_y f_\ell(x, y(x)) = 0$. Differentiating w.r.t. x gives

$$\nabla_{yx}^2 f_\ell(x, y(x)) + \nabla_{yy}^2 f_\ell(x, y(x)) \nabla y(x)^\top = 0$$

Computing the Adjoint Gradient (∇f)

$\nabla f(x)$ can be computed by applying the **chain rule** to $f_u(x, y(x))$

$$\nabla f(x) = \nabla_x f_u(x, y(x)) + \nabla y(x) \nabla_y f_u(x, y(x))$$

where $\nabla y(x)$ is the Jacobian of $y(x)$, typically **unknown**.

The **LL 1st-order optimality condition** is $\nabla_y f_\ell(x, y(x)) = 0$. Differentiating w.r.t. x gives

$$\nabla_{yx}^2 f_\ell(x, y(x)) + \nabla_{yy}^2 f_\ell(x, y(x)) \nabla y(x)^\top = 0$$

Assuming $\nabla_{yy}^2 f_\ell$ is **non-singular**,

$$\nabla y(x) = -\nabla_{xy}^2 f_\ell(x, y(x)) \nabla_{yy}^2 f_\ell(x, y(x))^{-1}$$

Adjoint Gradient

The **adjoint gradient** (or **hypergradient**) $\nabla f(x)$ is given by the **adjoint formula**

$$\nabla f(x) = \nabla_x f_u - \nabla_{xy}^2 f_\ell \nabla_{yy}^2 f_\ell^{-1} \nabla_y f_u$$

where all RHS gradients and Hessians are evaluated at $(x, y(x))$

Computing the Adjoint Gradient Inexactly

$$\nabla f(x) = \nabla_x f_u - \nabla_{xy}^2 f_\ell \nabla_{yy}^2 f_\ell^{-1} \nabla_y f_u$$

Two common approaches to compute $\nabla_{yy}^2 f_\ell^{-1} \nabla_y f_u$ are:

Computing the Adjoint Gradient Inexactly

$$\nabla f(x) = \nabla_x f_u - \nabla_{xy}^2 f_\ell \nabla_{yy}^2 f_\ell^{-1} \nabla_y f_u$$

Two common approaches to compute $\nabla_{yy}^2 f_\ell^{-1} \nabla_y f_u$ are:

1. Solve the adjoint equation

$$\nabla_{yy}^2 f_\ell \lambda = \nabla_y f_u$$

and compute the hypergradient as $\nabla f(x) \approx \nabla_x f_u - \nabla_{xy}^2 f_\ell \lambda$, where λ is computed inexactly.

Computing the Adjoint Gradient Inexactly

$$\nabla f(x) = \nabla_x f_u - \nabla_{xy}^2 f_\ell \nabla_{yy}^2 f_\ell^{-1} \nabla_y f_u$$

Two common approaches to compute $\nabla_{yy}^2 f_\ell^{-1} \nabla_y f_u$ are:

1. Solve the adjoint equation

$$\nabla_{yy}^2 f_\ell \lambda = \nabla_y f_u$$

and compute the hypergradient as $\nabla f(x) \approx \nabla_x f_u - \nabla_{xy}^2 f_\ell \lambda$, where λ is computed inexactly.

2. Approximate the inverse Hessian by a truncated Neumann series (assuming $\|\nabla_{yy}^2 f_\ell\| < 1$)

$$\nabla_{yy}^2 f_\ell^{-1} \nabla_y f_u \approx \sum_{i=0}^K (I - \nabla_{yy}^2 f_\ell)^i \nabla_y f_u$$

If $\|\nabla_{yy}^2 f_\ell\| < C_0$, apply the truncated Neumann series to $((1/C_0)\nabla_{yy}^2 f_\ell)^{-1}$

Bilevel Stochastic Gradient (BSG) Method

Algorithm BSG Method

Input: (x, y) , stepsize α

For $k = 0, 1, 2, \dots$ **do**

Step 1. Compute an approximate LL solution

$$\tilde{y} \simeq y(x)$$

Step 2. Compute a stochastic hypergradient direction using sampled gradients

$$d(x, \tilde{y}, \xi) \simeq -\nabla f(x)$$

Step 3. Update the UL variables (P_X is the projection operator)

$$x^+ = P_X(x + \alpha d(x, \tilde{y}, \xi))$$

End do

Outline

- 1 Introduction and motivation
- 2 Background: Bilevel stochastic gradient
- 3 Background: Derivative-free optimization with primitive directions
- 4 The BSGDFL algorithm
- 5 Application to neural architecture search

Black-box Mixed-Integer Optimization Problem

DFNDFL addresses the following **single-level** black-box mixed-integer optimization problem:

$$\begin{aligned} \min_{x \in \mathbb{R}^n} \quad & f(x) \\ \text{s.t.} \quad & x \in X \cap \mathcal{Z} \end{aligned}$$

where

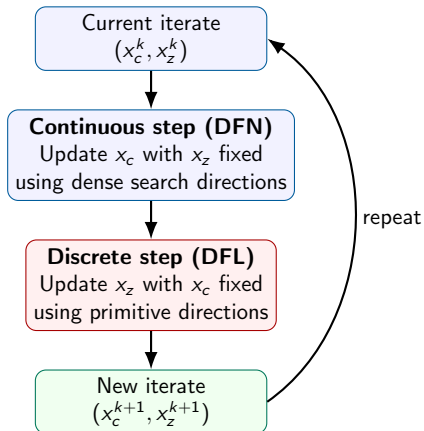
$f : \mathbb{R}^n \rightarrow \mathbb{R}$ is a **black-box** objective function

$X = \{x \in \mathbb{R}^n \mid \ell \leq x \leq u\}$ is the continuous feasible region

$\mathcal{Z} = \{x \in \mathbb{R}^n \mid x_i \in \mathbb{Z}, i \in I_z\}$ defines the integer variables

The objective function is unavailable analytically, and the integrality constraints cannot be relaxed.

DFNDFL: Alternating Strategy for Black-Box Mixed-Integer Optimization

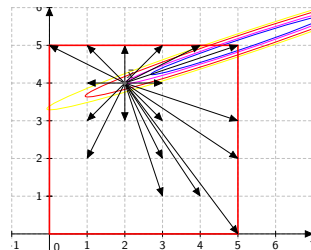


The discrete step performs a **discrete line search** to compute the largest feasible step along a primitive direction that satisfies a sufficient decrease condition.

Primitive directions

Definition (Primitive vector). A vector $v \in \mathbb{Z}^n$ is called primitive if $\text{GCD}(v_1, \dots, v_n) = 1$

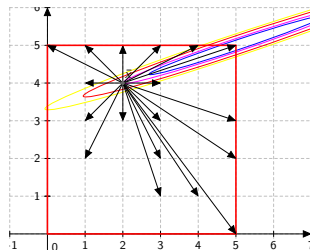
Primitive directions allow moving from any feasible integer point to any other without relaxing integrality.



Primitive directions

Definition (Primitive vector). A vector $v \in \mathbb{Z}^n$ is called primitive if $\text{GCD}(v_1, \dots, v_n) = 1$

Primitive directions allow moving from any feasible integer point to any other without relaxing integrality.



Definition (Set of feasible primitive directions). Given a point $x \in X \cap \mathbb{Z}$,

$$D^{\mathbb{Z}}(x) = \{p \in \mathbb{Z}^n : p_{\mathbb{Z}} \text{ is a primitive vector, } p_i = 0 \text{ for all } i \in I^c, \text{ and } x + p \in X \cap \mathbb{Z}\}$$

is the set of feasible primitive directions at x with respect to $X \cap \mathbb{Z}$

Discrete step (DFL)



Algorithm Discrete line search at x along direction p

Step 0. Compute the largest $\bar{\gamma}$ such that $x + \bar{\gamma}p \in X \cap \mathcal{Z}$

Step 1. If $\gamma > 0$ and $f(x + \gamma p) \leq f(x) - \eta$ then go to Step 2.

Else set $\gamma = 0$, return γ

Step 2. Set $\beta = \min\{\bar{\gamma}, 2\gamma\}$

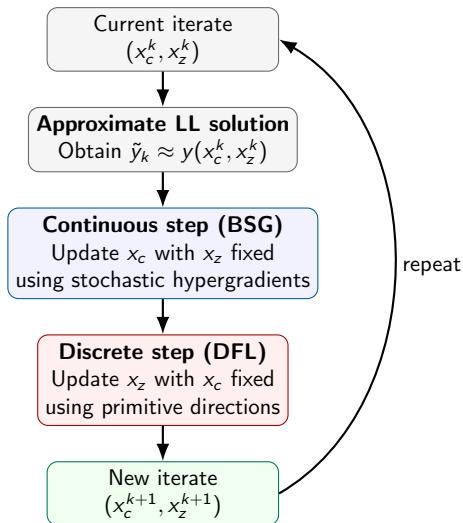
Step 3. If $f(x + \beta p) > f(x) - \eta$, return γ

Step 4. Set $\gamma = \beta$ and go to Step 2.

Outline

- 1 Introduction and motivation
- 2 Background: Bilevel stochastic gradient
- 3 Background: Derivative-free optimization with primitive directions
- 4 The BSGDFL algorithm**
- 5 Application to neural architecture search

BSGDFL: An Alternating Strategy for Mixed-Integer Bilevel Optimization



Continuous UL variables

Exploit bilevel structure through stochastic hypergradient information.

Integer UL variables

Preserve integrality using derivative-free line searches along primitive directions.

No relaxation or rounding of integer variables is required.

Continuous step (BSG)

This is essentially one iteration of the BSG method for fixed integer variables x_z^k

Continuous step (BSG)

This is essentially one iteration of the BSG method for fixed integer variables x_z^k

Step 1: Compute a [stochastic hypergradient direction](#)

$$d_k \simeq -\nabla_{x_c} f(x_c^k, x_z^k)$$

using $\tilde{y}_k \approx y(x_c^k, x_z^k)$ and

- inexact adjoint equation or truncated Neumann approximation
- stochastic gradients of f_u and f_ℓ

Continuous step (BSG)

This is essentially one iteration of the BSG method for fixed integer variables x_z^k

Step 1: Compute a [stochastic hypergradient direction](#)

$$d_k \simeq -\nabla_{x_c} f(x_c^k, x_z^k)$$

using $\tilde{y}_k \approx y(x_c^k, x_z^k)$ and

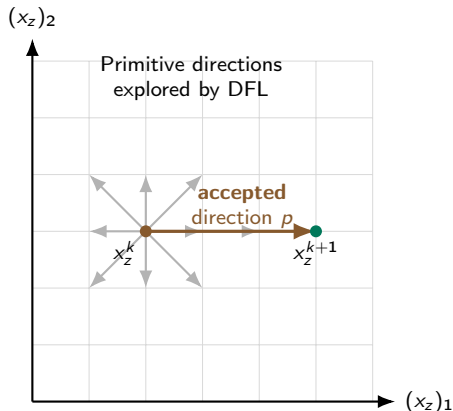
- inexact adjoint equation or truncated Neumann approximation
- stochastic gradients of f_u and f_ℓ

Step 2: Update the UL continuous variables:

$$(x_c^{k+1}, x_z^k) = (P_{X_c}(x_c^k + \alpha_k d_k), x_z^k)$$

Discrete step (DFL)

After the continuous step, DFL updates the integer variables while keeping x_c fixed.



Step 1: Explore primitive directions

Explore the current set of primitive directions D_k . For each $p \in D_k$, compute a stepsize γ using the discrete line search.

Step 2: Enrich the search

If no direction is accepted, reduce the sufficient decrease threshold ($\eta_{k+1} = \rho\eta_k$), and add new primitive directions: $D_{k+1} \supset D_k$

Step 3: Update UL integer variables

$$(x_c^{k+1}, x_z^{k+1}) = (x_c^{k+1}, x_z^k + \gamma p)$$

Outline

- 1 Introduction and motivation
- 2 Background: Bilevel stochastic gradient
- 3 Background: Derivative-free optimization with primitive directions
- 4 The BSGDFL algorithm
- 5 Application to neural architecture search

Baselines: DARTS and StocBiO

We apply both methods to the continuous relaxation of the NAS problem (each discrete operation choice is relaxed into a softmax mixture over candidate operations) and round to a discrete architecture *after* the search is complete.

DARTS (Liu et al., 2019).

DARTS approximates the hypergradient by applying the chain rule to $f_u(x, y - \eta \nabla_y f_\ell(x, y))$ instead of $f_u(x, y(x))$

Not guaranteed to be a descent direction.

StocBiO (Ji et al., 2020).

Approximates the inverse Hessian with a truncated Neumann series and automatic differentiation.

Avoids second-order Hessian computation entirely.

Experimental setup

Dataset and search space. CIFAR-10, split into 25,000 training and 25,000 validation images

- Depth per cell: $\{4, 5, 6, 7, 8, 10\}$
- Width per cell: $\{32, 48, 64, 96, 128\}$
- Continuous UL variables: weight decay, label smoothing

All results are averaged over 10 independent runs

Training and tuning. LL problem optimized with SGD, minibatch size 64

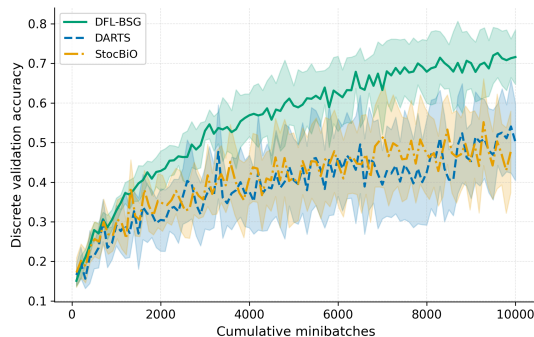
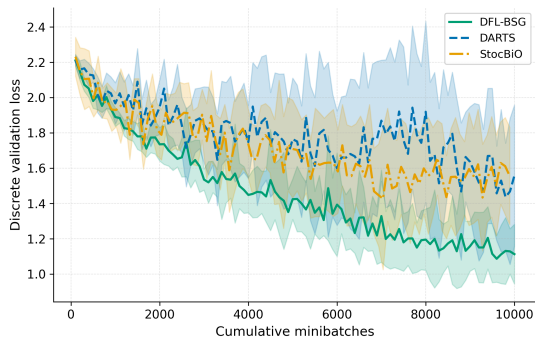
- Grid search over UL and LL stepsizes
- BSGDFL stepsizes for integer variables: $\{1, 3, 5\}$
- DLS with $\eta = 10^{-3}$, initial $\gamma = 4$

Results: Accuracy and search cost

Method	Test accuracy	GPU-hours
DARTS	93.90% $\pm$ 0.40%	3.78 $\pm$ 0.01
StocBiO	94.36% $\pm$ 0.27%	0.87 $\pm$ 0.01
BSGDFL	94.46% $\pm$ 0.68%	1.14 $\pm$ 0.05

BSGDFL achieves the highest test accuracy among the three methods, with a roughly $3\times$ lower search cost than DARTS, but a higher search cost than StocBiO.

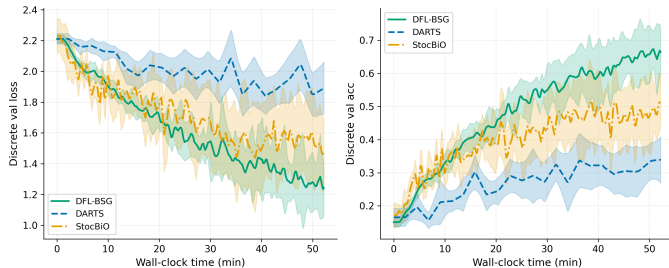
Results: Validation loss and accuracy



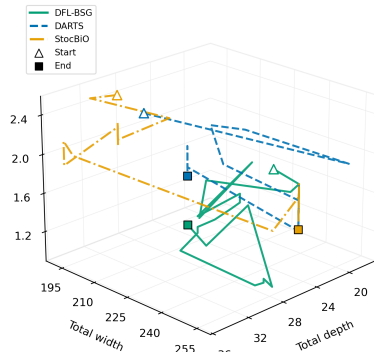
Validation loss (left) and accuracy (right) over cumulative minibatches, evaluated on the rounded architecture, averaged over 10 runs with 95% t -intervals shown.

BSGDFL ("DFL-BSG" in the figures) reduces the discrete validation loss monotonically, while DARTS and StocBiO plateau at a higher level.

Results: Architecture trajectories



Validation loss (left) and accuracy (right) over wall-clock time, evaluated on the rounded architecture



Validation loss over architecture depth and width

BSGDFL ("DFL-BSG" in the figures) favors exploitation of promising discrete architectures, while the two baselines favor exploration.

Thank you

Questions and discussion

University of Cincinnati

`gareeis@mail.uc.edu` `giovanto@ucmail.uc.edu`